Reversible Adders in Vivado: Procedure, Reports and Screenshots

1. What you need

- **Vivado:** version 2022.2 or later. The free Standard edition is enough; it includes the Artix-7 part xc7a35tcpg236-1 used here.
- **Folder:** unzip this package to a path without spaces, for example `C:/rev/vivado`. Spaces in the path break the Tcl scripts.
- **Time:** about 30 minutes for all simulations, and 20 to 60 minutes for synthesis of the eight 32-bit modules.

Nothing needs to be typed into Verilog files. All the code is generated and was checked with Icarus Verilog: every testbench passes.

2. Create the project

1. Start Vivado.
2. Choose **Tools > Run Tcl Script…** and select `create_project.tcl`.
3. The project `rev_adders` opens. It is created in the folder `rev_adders_project` inside your package folder.

The Sources window now shows:

- `rev_gates.v`, the gate library: NOT, Feynman, Toffoli, Peres, TSG and R2.
- `rev_adders.v`, the five 4-bit modules of Section 5 of the paper.
- `rev_adders_wide.v`, 56 modules: seven designs × 4, 8, 16 and 32 bits × as built and simplified.
- Under *Simulation Sources*, seven testbenches.

Screenshot 1, Project overview: the Vivado window with the Sources hierarchy expanded.

After restarting Vivado, open the project again before anything else: **File > Open Project** and choose `rev_adders_project/rev_adders.xpr`, or type `open_project C:/rev/vivado/rev_adders_project/rev_adders.xp`
The scripts below also open it themselves if needed.

Typing commands: in the Tcl Console, a script is run with `source` and its full path, with forward slashes, for example `source C:/rev/vivado/run_all_tests.tcl`. Typing only the file name gives *invalid command name*.

3. Run the simulations

3.1 One testbench

1. In the Flow Navigator choose **SIMULATION > Run Simulation > Run Behavioral Simulation**. This runs `tb_rev_adders`, the 4-bit adders of Section 5.
2. Read the **Tcl Console** at the bottom. Each module prints one line, then `ALL TESTS PASSED`:

```
cla_as_built          wrong answers: 0 of 516
...
ALL TESTS PASSED
```

Vivado may open the testbench file at the `$finish` line when the run ends. This is normal.

3.2 Another testbench

1. In Sources, open *Simulation Sources* > *sim_1*, right-click `tb_wide_32` and choose **Set as Top**.
2. Run Behavioral Simulation again.

Testbench	What it checks	Expected end
tb_rev_gates	every gate on every input	ALL GATE TESTS PASSED
tb_rev_adders	4-bit adders of Section 5, all 512 inputs + 4 named cases	ALL TESTS PASSED
tb_wide_4	fourteen 4-bit modules, all 512 inputs	ALL TESTS PASSED
tb_wide_8, _16, _32	fourteen modules each, 900 vectors	ALL TESTS PASSED
tb_demo_8	four named 8-bit additions on four designs	DEMO PASSED

3.3 All testbenches at once

Choose **Tools > Run Tcl Script…** and select `run_all_tests.tcl`. The seven testbenches run one after the other, and each prints its result in the Tcl Console.

> **Screenshot 2, Verification:** the Tcl Console showing the 14 lines of `tb_wide_32` and `ALL TESTS PASSED`.

4. Waveform screenshots

4.1 Four designs side by side (recommended)

1. Set `tb_demo_8` as top and run Behavioral Simulation.
2. In the waveform window, click **Zoom Fit**, the button with four outward arrows.
3. Select `a`, `b`, `expected` and the four `sum_*` signals. Right-click and choose **Radix > Unsigned Decimal**.
4. Each 50 ns step now shows one addition, with the expected value and the result of each design:

Time	Addition	Note
0–50 ns	100 + 27 + 0 = 127	
50–100 ns	255 + 1 + 0 = 256	carry through all 8 bits, cout = 1
100–150 ns	170 + 85 + 1 = 256	alternating bits with carry input
150–200 ns	200 + 200 + 1 = 401	

The Draper adder has no carry input, so it shows a + b: 255 and 400 in the last two steps. Its `clean_draper` signal stays 1, which means every ancilla line returned to 0.

> **Screenshot 3, Waveform:** all signals in decimal over 0–200 ns.

4.2 The 4-bit adders of Kumar et al.

1. Set `tb_rev_adders` as top and run the simulation.
2. Zoom to 0–200 ns.
3. To show one adder's internal lines, expand `dut1` … `dut5` in the **Scope** window, select a module, and drag signals from the **Objects** window into the waveform.

Screenshot 4, Waveform of the 4-bit adders: the four named cases (5+3, 15+1, 10+5, 9+9+1).

How to capture: on Windows, press **Win + Shift + S** and select the area. Use a light theme for figures: Tools > Settings > Display > Color theme.

5. Schematic screenshots

1. Choose **Tools > Run Tcl Script…** and select `screenshot_schematic.tcl`. It opens the RTL schematic of `hybrid_8_simplified`.
2. For another module, edit the `set top` line first. Small modules give readable pictures, for example `cla_blocks_4_simplified` or `hybrid_4_simplified`.
3. In the GUI you can do the same: right-click a module in Sources, choose **Set as Top**, then **RTL ANALYSIS > Open Elaborated Design > Schematic**.

Each box is one reversible gate instance (Peres, Feynman, Toffoli and so on) from the gate library, so the schematic shows the same structure as the circuit figures of the paper.

Screenshot 5, Schematic: the elaborated design of `hybrid_8_simplified` or `cla_blocks_4_simplified`.

6. Synthesis reports: LUTs, delay and power

6.1 All designs in one run

1. Open `synth_all.tcl` in a text editor if you want to change the module list or set `implement 0` for synthesis only. The default is all seven designs at 32 bits, simplified, plus the CLA blocks as built.
2. With the project open, choose **Tools > Run Tcl Script…** and select `synth_all.tcl`.
3. When it finishes, open the folder `reports`:

File	Content
`summary.csv`	one row per module: LUTs, input-to-output delay (ns), logic levels, power (W). Open it in Excel.
`<module>_utilization.txt`	full resource report (Slice LUTs, etc.)
`<module>_timing.txt`	the longest input-to-output path, gate by gate
`<module>_power.txt`	Vivado's vectorless power estimate

The modules are synthesized *out of context*: a 32-bit module has 98 ports, so no package pins are used. With `implement 1`, the design is also placed and routed, and the delay is post-route.

6.2 One module in the GUI

1. Set the module as top.
2. Run `synth_one.tcl`, or choose **SYNTHESIS > Run Synthesis** and then **Open Synthesized Design**.
3. Use **Report Utilization** and **Report Timing Summary**.

Screenshot 6, Reports: the Utilization summary and the timing path of `hybrid_32_simplified`, or `summary.csv` opened in Excel.

7. How to use these results honestly

- **Simulation results:** every adder computes a + b (+ cin) correctly. You can state this in the thesis and the paper; it is an independent check of the decision-diagram proofs.
- **Synthesis results** (LUTs, ns, W) describe an ordinary FPGA circuit for the same addition. Vivado re-optimizes the logic, so as-built and simplified versions often give similar numbers.

- **Not reversible-logic measures:** they do not measure quantum cost, T-count or ancilla lines. They are not physical results of a reversible or quantum implementation, and they are no evidence of lower energy.
- **In the thesis:** present them in a separate table titled, for example, *"FPGA implementation of the generated netlists (Artix-7, Vivado)"*, with this explanation.
- **In the journal paper:** do not present them as savings. Section 9 of the paper states that no area, delay or power figure is reported.
- **Power:** the power figure is a vectorless estimate with default switching activity, and on a small design it is dominated by static power. Say so wherever you show it.

8. Problems and fixes

Problem	Fix
"No HDL sources found" or a different part (xc7vx485t) in the log	Vivado was restarted without the project; open `rev_adders.xpr` first
"Spawn failed: No error" during simulation	Windows could not start the compiler while Vivado re-parsed the sources; `run_all_tests.tcl` retries automatically, or run that testbench again
"couldn't read file" when running a script	the folder path has spaces, or the file moved; unzip again to `C:/rev/vivado`
Simulation stops at 1000 ns without PASSED	the wrong top is set; choose the testbench, not an adder module, as top of sim_1
`synth_all.tcl` says "no open project"	run `create_project.tcl` first, or open `rev_adders_project/rev_adders.xpr`
Part xc7a35tcpg236-1 not installed	change `set part` in `create_project.tcl` to any installed 7-series part
Implementation complains about I/O pins	use `synth_all.tcl` or `synth_one.tcl`; both use out-of-context mode